

ТЕХНИЧЕСКИЙ ГАЙД: ЛОКАЛЬНАЯ УСТАНОВКА И АДАПТАЦИЯ ПО ДЛЯ КОНТЕНТ-ЗАВОДА

1. ВВЕДЕНИЕ

Данный гайд описывает, как заменить облачные платные сервисы из оригинальной стратегии на локальные или российские аналоги без потери функциональности контент-завода.

1.1. Цели локализации

- Снижение ежемесячных затрат с \$146 до \$20-40
- Независимость от зарубежных сервисов (санкции, блокировки)
- Полный контроль над данными и процессами
- Возможность работы без интернета (для части операций)

1.2. Системные требования

Компонент	Требование
ОС	Windows 10 Pro (64-bit) с WSL2
CPU	Intel Core i5 8-го поколения или AMD Ryzen 5 3600+
RAM	16 GB (рекомендуется 32 GB для локальных LLM)
GPU	NVIDIA RTX 3060 12GB или выше
Диск	100 GB свободного места (SSD предпочтительно)
Интернет	Для API социальных сетей и облачных сервисов

Table 1: Минимальные системные требования

2. СРАВНИТЕЛЬНАЯ ТАБЛИЦА ПО

2.1. LLM и текстовые модели

Инструмент	Тип	Локальная установка	Стоимость	Рекомендация
OpenAI GPT-4/5	Облако (SaaS)	Нет	\$40/мес	DeepSeek локально (\$0)
Claude (Anthropic)	Облако (SaaS)	Нет	\$30/мес	GigaChat (бесплатно)
DeepSeek R1/V3	Облако + Open Source	Да (WSL2, Ollama)	\$0 локально	Ollama (основной)
YandexGPT 4	Облако (РФ)	Нет (только API)	~\$10/мес	Использовать для скорости

Table 2: Сравнение LLM моделей

2.2. Оркестрация и автоматизация

Make.com vs n8n:

- Было: Make.com — облако, \$16/мес, лимит 10,000 операций
- Стало: n8n — self-hosted через Docker, бесплатно, без лимитов
- Совместимость: Концепция узлов идентична, миграция занимает 2-3 часа

2.3. Генерация изображений

Инструмент	Локальный аналог	Было	Стало	Экономия
DALL-E 3	Stable Diffusion XL	\$0.04-0.08/шт	\$0	100%
Midjourney	SD + Realistic Vision	\$10-30/мес	\$0	\$30/мес
Flux	Flux Dev (ComfyUI)	\$0.03/шт	\$0	100%

Table 3: Сравнение инструментов генерации изображений

2.4. Итоговая стоимость после локализации

Компонент	Было	Стало	Экономия
LLM (GPT-4/5)	\$40/мес	\$0	-\$40
Make.com	\$16/мес	\$0	-\$16
DALL-E/Midjourney	\$30/мес	\$0	-\$30
Sora/Kling	\$50/мес	\$15/мес	-\$35
YandexGPT	\$10/мес	\$10/мес	\$0
ИТОГО	\$146/мес	\$25/мес	-\$121 (-83%)

Table 4: Финансовый эффект локализации

3. ЛОКАЛЬНАЯ УСТАНОВКА LLM

3.1. Вариант 1: DeepSeek через Ollama (рекомендуется)

3.1.1. Преимущества

- Простая установка (один установщик)
- Автоматическое управление моделями
- Совместимый OpenAI API endpoint
- Работает на CPU и GPU

3.1.2. Установка Ollama на Windows

Шаг 1: Скачать установщик

Скачать с официального сайта: <https://ollama.com/download/windows>

Или через PowerShell (от имени администратора):

```
Invoke-WebRequest -Uri https://ollama.com/download/OllamaSetup.exe -OutFile  
OllamaSetup.exe  
.\OllamaSetup.exe
```

Шаг 2: Скачать модель DeepSeek R1

```
ollama pull deepseek-r1:7b
```

Для более мощного варианта (требуется 32GB RAM):

```
ollama pull deepseek-r1:14b
```

3.1.3. Размеры моделей

Модель	Размер	Минимум RAM
deepseek-r1:7b	4.3 GB	8 GB
deepseek-r1:14b	8.6 GB	16 GB
deepseek-r1:32b	20 GB	32 GB

Table 5: Требования к памяти для моделей DeepSeek

3.1.4. Запуск локального API-сервера

Ollama автоматически запускает сервер на <http://localhost:11434>

Проверить статус:

```
ollama list
```

3.1.5. Тест API

Пример Python-скрипта для тестирования:

```
import requests
import json

url = "http://localhost:11434/api/chat"

payload = {
    "model": "deepseek-r1:7b",
    "messages": [
        {
            "role": "system",
            "content": "Ты — эксперт по созданию контента."
        },
        {
            "role": "user",
            "content": "Напиши короткий пост о пользе массажа."
        }
    ],
    "stream": False
}

response = requests.post(url, json=payload)
result = response.json()
print(result['message']['content'])
```

Запуск теста:

```
python test_ollama.py
```

3.1.6. Интеграция в проект

Обновление файла .env:

```
OLLAMA_API_URL=http://localhost:11434/api
OLLAMA_MODEL=deepseek-r1:7b
YANDEX_GPT_API_KEY=...
```

Обновление scripts/content_generator.py:

```
import os
import requests
from dotenv import load_dotenv

load_dotenv()
```

```
OLLAMA_URL = os.getenv('OLLAMA_API_URL')
OLLAMA_MODEL = os.getenv('OLLAMA_MODEL')

def generate_text_ollama(system_prompt, user_prompt):
    payload = {
        "model": OLLAMA_MODEL,
        "messages": [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ],
        "stream": False,
        "options": {
            "temperature": 0.7,
            "top_p": 0.9
        }
    }
```

```
response = requests.post(OLLAMA_URL, json=payload, timeout=120)

if response.status_code == 200:
    result = response.json()
    return result['message']['content']
else:
    raise Exception(f'Ollama API error: {response.status_code}')
```

3.2. Вариант 2: DeepSeek через LM Studio

3.2.1. Преимущества

- Графический интерфейс (удобнее для новичков)
- Встроенный чат для тестирования
- Автоматическая настройка GPU

3.2.2. Шаги установки

1. Скачать LM Studio с <https://lmstudio.ai/>
2. Запустить LM Studio → Search Models → Найти "deepseek-r1"
3. Скачать модель (например, deepseek-r1-7b-GGUF)
4. Local Server → Start Server (порт: 1234)
5. Тестировать через API на <http://localhost:1234/v1>

3.2.3. Тест API

API совместим с OpenAI:

```
import openai
```

```
openai.api_key = "lm-studio"
```

```
openai.api_base = "http://localhost:1234/v1"
```

```

response = openai.ChatCompletion.create(
    model="deepseek-r1-7b",
    messages=[
        {"role": "system", "content": "Ты — эксперт."},
        {"role": "user", "content": "Напиши пост о массаже."}
    ],
    temperature=0.7
)

print(response['choices'][0]['message']['content'])

```

3.3. Вариант 3: YandexGPT 4 (облако РФ)

3.3.1. Преимущества

- Не требует мощного железа
- Хорошее понимание русского языка
- Низкая стоимость (~\$10/мес до 1М токенов)

3.3.2. Шаги настройки

1. Создать аккаунт в Yandex Cloud: <https://cloud.yandex.ru/>
2. Создать Folder (каталог) в Console
3. Получить API-ключ: <https://cloud.yandex.ru/docs/iam/operations/api-key/create>
4. Включить биллинг (минимальный платеж: 1000₽)

3.3.3. Тест API

```

import requests
import os

YANDEX_API_KEY = os.getenv('YANDEX_GPT_API_KEY')
YANDEX_FOLDER_ID = os.getenv('YANDEX_FOLDER_ID')

def generate_text_yandex(prompt, system_prompt="Ты — эксперт."):
    url = "https://llm.api.cloud.yandex.net/foundationModels/v1/completion"

```

```

    headers = {
        "Authorization": f"Api-Key {YANDEX_API_KEY}",
        "Content-Type": "application/json"
    }

    payload = {
        "modelUri": f"gpt://{YANDEX_FOLDER_ID}/yandexgpt/latest",
        "completionOptions": {
            "stream": False,
            "temperature": 0.7,
            "maxTokens": 2000
        },

```

```
"messages": [
    {"role": "system", "text": system_prompt},
    {"role": "user", "text": prompt}
]
}

response = requests.post(url, headers=headers, json=payload)

if response.status_code == 200:
    result = response.json()
    return result['result']['alternatives'][0]['message']['text']
else:
    raise Exception(f"YandexGPT error: {response.status_code}")
```

3.4. Сравнение вариантов LLM

Критерий	Ollama	LM Studio	YandexGPT
Стоимость	Бесплатно	Бесплатно	~\$10/мес
Железо	16GB RAM, GPU	16GB RAM, GPU	Нет (облако)
Скорость	Средняя	Средняя	Быстро
Качество (RU)	Хорошее	Хорошее	Отличное
Приватность	100% локально	100% локально	Облако (РФ)
Настройка	Низкая	Очень низкая	Средняя

Table 6: Сравнение LLM решений

Итоговая рекомендация:

- Для разработки: Ollama (DeepSeek R1:7b)
- Для продакшена: Гибрид — 70% Ollama + 30% YandexGPT

4. ЗАМЕНА [MAKE.COM](#) НА N8N

4.1. Что такое n8n

n8n — open-source аналог [Make.com/Zapier](https://make.com/) с возможностями:

- Визуальный конструктор workflow (drag-and-drop)
- 400+ встроенных интеграций
- Self-hosted развертывание (Docker, VPS, локально)
- Совместимый API (HTTP, Webhook, JavaScript)

Официальный сайт: <https://n8n.io/>

4.2. Установка n8n на Windows 10

4.2.1. Вариант 1: Docker Desktop (рекомендуется)

Шаг 1: Установить Docker Desktop

Скачать с <https://www.docker.com/products/docker-desktop/>

Шаг 2: Создать docker-compose.yml

```
version: '3.8'

services:
  n8n:
    image: n8nio/n8n:latest
    container_name: n8n
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      - N8N_BASIC_AUTH_ACTIVE=true
      - N8N_BASIC_AUTH_USER=admin
      - N8N_BASIC_AUTH_PASSWORD=your_password
      - N8N_HOST=localhost
      - N8N_PORT=5678
      - WEBHOOK_URL=http://localhost:5678/
      - GENERIC_TIMEZONE=Europe/Moscow
    volumes:
      - n8n_data:/home/node/.n8n

volumes:
  n8n_data:
```

Шаг 3: Запустить n8n

```
docker-compose up -d
docker ps
```

Открыть в браузере: <http://localhost:5678>

Логин: admin / Пароль: your_password

4.2.2. Вариант 2: Установка через npm

Требования: Node.js 18+

```
npm install n8n -g  
n8n start
```

4.3. Миграция workflow из [Make.com](#)

4.3.1. Экспорт из [Make.com](#)

1. Открыть [Make.com](#) → Scenarios
2. Выбрать workflow → три точки → Export blueprint (JSON)
3. Сохранить файл

4.3.2. Анализ структуры

[Make.com](#) использует:

- Modules (блоки)
- Connections (связи)
- Mapping (передача данных)

n8n использует:

- Nodes (блоки)
- Connections (связи)
- Expressions ({{ \$json.field }})

4.3.3. Создание workflow в n8n

Пример: Workflow для SEO-статей

Структура [Make.com](#):

[Schedule] → [Google Sheets] → [YandexGPT] → [GPT-4] →
[DALL-E] → [Sheets Update] → [WordPress]

Структура n8n (аналог):

1. **Cron Node:** Триггер по расписанию (каждые 2 дня в 10:00)
2. **Google Sheets Node:** Чтение данных (операция: Get Rows)
3. **HTTP Request Node (YandexGPT):** Генерация ключевых запросов
4. **HTTP Request Node (Ollama):** Генерация SEO-статьи через локальный DeepSeek
5. **Function Node:** Обработка и форматирование данных
6. **Google Sheets Node:** Запись результата
7. **WordPress Node:** Публикация поста

4.4. Конфигурация n8n для локальных сервисов

Создать .env файл:

```
N8N_BASIC_AUTH_USER=admin  
N8N_BASIC_AUTH_PASSWORD=secure_password_123  
N8N_HOST=localhost  
N8N_PORT=5678
```

OLLAMA_API_URL=http://host.docker.internal:11434/api/chat
STABLE_DIFFUSION_API_URL=http://host.docker.internal:7860

YANDEX_GPT_API_KEY=AQVNxxxxxxxxxx
YANDEX_FOLDER_ID=b1gxxxxxxxxxx
GOOGLE_SHEET_ID=1aBcDeFgHiJkLmNoPqRsTuVwXyZ...

WORDPRESS_URL=https://entuziastov75.ru
WORDPRESS_USER=admin
WORDPRESS_APP_PASSWORD=xxxx xxxx xxxx xxxx

Примечание: host.docker.internal — специальный DNS-адрес для доступа из Docker-контейнера к локальным сервисам на Windows.

4.5. Сравнение [Make.com](#) и n8n

Критерий	Make.com	n8n
Стоимость	\$16-29/мес	Бесплатно (self-hosted)
Лимиты	10,000 операций/мес	Без лимитов
Хостинг	Только облако	Self-hosted или облако
Интеграции	1500+	400+ (расширяется)
Сложность	Низкая (GUI)	Средняя (GUI + сервер)
Приватность	Данные в облаке	100% контроль
Кастомизация	Ограничена	Полная (JS-код)

Table 7: Make.com vs n8n

5. ЛОКАЛЬНАЯ ГЕНЕРАЦИЯ ИЗОБРАЖЕНИЙ

5.1. Установка Stable Diffusion WebUI (AUTOMATIC1111)

5.1.1. Требования

- NVIDIA GPU с 6GB+ VRAM (минимум GTX 1660)
- Python 3.10.x
- Git

5.1.2. Шаг 1: Установить Python 3.10.6

Скачать с <https://www.python.org/downloads/release/python-3106/>

Важно: Отметить "Add Python to PATH" при установке.

5.1.3. Шаг 2: Установить Git

Скачать с <https://git-scm.com/download/win>

5.1.4. Шаг 3: Клонировать репозиторий

```
cd C:\Projects\content-factory-entusiast
git clone https://github.com/AUTOMATIC1111/stable-diffusion-webui.git
cd stable-diffusion-webui
```

5.1.5. Шаг 4: Скачать модель Stable Diffusion

Рекомендуемая модель: Realistic Vision v5.1

1. Перейти на Civitai: <https://civitai.com/models/4201/realistic-vision-v51>
2. Скачать файл realisticVisionV51_v51VAE.safetensors (~2-5 GB)
3. Поместить в папку: stable-diffusion-webui/models/Stable-diffusion/

5.1.6. Шаг 5: Запустить WebUI

```
.\webui.bat --api --xformers
```

Параметры:

- `--api`: включить REST API на порту 7860
- `--xformers`: оптимизация для NVIDIA GPU (быстрее)

Первый запуск займет 5-10 минут.

Открыть в браузере: <http://localhost:7860>

5.1.7. Шаг 6: Тест генерации изображения

Через GUI:

1. Открыть <http://localhost:7860>
2. Вкладка txt2img
3. Промпт: "Professional massage room in modern wellness center, cozy atmosphere, wooden interior, warm lighting, photorealistic style, high quality, 4K"
4. Negative Prompt: "blurry, low quality, distorted, text, watermark, cartoon, anime"
5. Settings: Width 1080, Height 1080, Steps 30, CFG Scale 7, Sampler DPM++ 2M Karras
6. Нажать Generate

Через API (Python):

```
import requests
import base64
import os
```

```
SD_API_URL = 'http://127.0.0.1:7860'
```

```
def generate_image_sd(prompt, negative_prompt="",
width=1080, height=1080):
    payload = {
        "prompt": prompt,
        "negative_prompt": negative_prompt,
        "width": width,
        "height": height,
        "steps": 30,
        "sampler_name": "DPM++ 2M Karras",
        "cfg_scale": 7,
        "seed": -1
    }
```

```
    response = requests.post(
        f'{SD_API_URL}/sdapi/v1/txt2img',
        json=payload,
        timeout=300
    )

    if response.status_code == 200:
        r = response.json()
        image_data = base64.b64decode(r['images'][0])

        output_path = f'data/generated/images/sd_{hash(prompt)}.png'
        os.makedirs(os.path.dirname(output_path), exist_ok=True)

        with open(output_path, 'wb') as f:
            f.write(image_data)

        print(f'Image generated: {output_path}')
        return output_path
    else:
        raise Exception(f'SD API error: {response.status_code}')
```

5.2. Интеграция Stable Diffusion с n8n

Создать Flask-сервер для webhook:

```
from flask import Flask, request, jsonify
import sd_image_generator

app = Flask(name)
```

```
@app.route('/generate_image', methods=['POST'])
def generate_image_endpoint():
    data = request.json
    prompt = data.get('prompt')
    negative_prompt = data.get('negative_prompt', 'blurry, low quality')
    width = data.get('width', 1080)
    height = data.get('height', 1080)
```

```
    try:
        image_path = sd_image_generator.generate_image_sd(
            prompt, negative_prompt, width, height
        )

        public_url = f'http://localhost:8000/{image_path}'

        return jsonify({
            'success': True,
            'image_url': public_url,
            'local_path': image_path
        })
    except Exception as e:
        return jsonify({
            'success': False,
            'error': str(e)
        }), 500
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5001)
```

Запустить сервер:

```
python scripts/sd_api_server.py
```

В другом терминале запустить ngrok для публичного URL:

```
ngrok http 5001
```

В n8n workflow: Добавить HTTP Request Node с URL от ngrok.

5.3. Альтернатива: Kandinsky 3.1

Kandinsky 3.1 — российская модель от Sber AI.

API: <https://fusionbrain.ai/>

Преимущества:

- Российская разработка (Sber AI)
- Бесплатный доступ (с лимитами)
- Хорошее качество изображений

Минусы:

- Облачный API (нет локальной установки)
- Лимиты на количество запросов

6. АДАПТАЦИЯ АРХИТЕКТУРЫ ПРОЕКТА

6.1. Новая архитектура с локальными сервисами

Компоненты системы:

1. **Хранилище данных:** Google Sheets (5 листов) — без изменений
2. **Оркестратор:** n8n (Docker на Windows, бесплатно) — заменяет [Make.com](https://make.com)
3. **LLM движки:** 70% DeepSeek (Ollama, локально) + 30% YandexGPT (облако РФ)
4. **Генерация изображений:** Stable Diffusion XL (локально) — заменяет DALL-E
5. **Генерация видео:** Kling AI (облако) — без изменений
6. **Обработка медиа:** FFmpeg + Whisper (локально)
7. **Публикация:** 10+ платформ через API — без изменений

6.2. Обновленная структура проекта

content-factory-entusiast/

```
|— .env
|— .gitignore
|— config/
|   |— platforms.json
|   |— prompts.json
|   |— tone_of_voice.md
|   |— services_endpoints.json
|— data/
|   |— услуги.csv
|   |— темы_контента.csv
|   |— generated/
|   |— cache/
|— scripts/
|   |— data_sync.py
|   |— content_generator.py
|   |— sd_image_generator.py
|   |— sd_api_server.py
|   |— video_processor.py
```

```
| |— subtitle_generator.py
| |— publisher.py
| |— backup.py
| — n8n/
| |— docker-compose.yml
| |— workflows/
| |— credentials.json
| — stable-diffusion-webui/
| — assets/
| — docs/
```

7. ИЗМЕНЕНИЯ В WORKFLOW

7.1. Workflow для SEO-статей (адаптированный)

Было (Make.com + OpenAI):
[Schedule] → [Google Sheets] → [YandexGPT] → [GPT-4] →
[DALL-E] → [Google Sheets] → [WordPress]

Стало (n8n + Ollama + Stable Diffusion):
[Cron] → [Google Sheets] → [YandexGPT] → [Ollama DeepSeek] →
[SD WebUI] → [Google Sheets] → [WordPress]

7.2. Ключевые изменения

Компонент	Было	Стало
Триггер	Schedule	Cron (аналог)
LLM API	https://api.openai.com/v1	http://localhost:11434
Image API	https://api.openai.com/v1/images	http://localhost:7860
Оркестратор	https://www.make.com/	http://localhost:5678

Table 8: URL замены для workflow

8. КОНФИГУРАЦИОННЫЕ ФАЙЛЫ

8.1. config/services_endpoints.json

```
{
  "llm": {
    "primary": {
      "name": "Ollama DeepSeek",
      "url": "http://localhost:11434/api/chat",
      "model": "deepseek-r1:7b",
      "type": "local",
      "cost_per_1k_tokens": 0
    },
    "secondary": {
      "name": "YandexGPT 4",
      "url": "https://llm.api.cloud.yandex.net/...",

```

```

"model": "yandexgpt/latest",
"type": "cloud",
"cost_per_1k_tokens": 0.01
},
"image_generation": {
"primary": {
"name": "Stable Diffusion XL",
"url": "http://localhost:7860/sdapi/v1/txt2img",
"type": "local",
"cost_per_image": 0
}
}
}

```

8.2. config/prompts.json

Промпт для SEO-статей:

```

{
"seo_article": {
"system": "Ты — SEO-копирайтер для Центра здоровья  
'Энтузиаст' на Крайнем Севере (Ноябрьск, ЯНАО).",
"user_template": "Напиши SEO-оптимизированную статью  
на тему: '{topic}'."
}
}

```

9. РАЗВЕРТЫВАНИЕ НА WINDOWS 10

9.1. Полная последовательность установки

9.1.1. Шаг 1: Подготовка системы

1. Обновить Windows 10 до последней версии
2. Включить WSL2: wsl --install (перезагрузить)
3. Установить Docker Desktop с поддержкой WSL 2

9.1.2. Шаг 2: Установка Python и зависимостей

Установить Python 3.10.6

<https://www.python.org/downloads/release/python-3106/>

```
python --version
```

```

pip install openai requests pandas python-dotenv
google-api-python-client Pillow flask faster-whisper

```


9.1.3. Шаг 3: Установка Ollama (DeepSeek)

Скачать:

<https://ollama.com/download/windows>

Запустить OllamaSetup.exe

```
ollama pull deepseek-r1:7b  
ollama list
```

9.1.4. Шаг 4: Установка Stable Diffusion WebUI

```
cd C:\Projects  
git clone https://github.com/AUTOMATIC1111/stable-diffusion-webui.git  
cd stable-diffusion-webui
```

Скачать модель Realistic Vision v5.1

Поместить в models\Stable-diffusion

```
.\webui.bat --api --xformers
```

Первый запуск займет 5-10 минут.

9.1.5. Шаг 5: Установка n8n (Docker)

```
mkdir C:\Projects\content-factory-entusiast\n8n  
cd C:\Projects\content-factory-entusiast\n8n
```

Создать docker-compose.yml

```
docker-compose up -d  
docker ps
```

Открыть: <http://localhost:5678>

9.1.6. Шаг 6: Настройка проекта

```
cd C:\Projects  
git clone <repo-url> content-factory-entusiast  
cd content-factory-entusiast
```

Создать .env файл и заполнить API-КЛЮЧИ

```
python scripts/data_sync.py
python scripts/sd_api_server.py
```

9.1.7. Шаг 7: Автоматизация запуска

Создать start_all_services.bat:

```
@echo off
echo Starting Content Factory Services...

echo [1/4] Starting Ollama...
REM Ollama запускается автоматически

echo [2/4] Starting Stable Diffusion WebUI...
start /B cmd /c "cd C:\Projects\stable-diffusion-webui &&
webui.bat --api --xformers"

echo [3/4] Starting n8n...
start /B cmd /c "cd C:\Projects\content-factory-entusiast\n8n &&
docker-compose up"

echo [4/4] Starting Flask API Server...
start /B cmd /c "cd C:\Projects\content-factory-entusiast &&
python scripts/sd_api_server.py"

echo All services started!
pause
```

Добавить в автозагрузку Windows:

1. Нажать Win + R
2. Ввести: shell:startup
3. Создать ярлык для start_all_services.bat

10. TROUBLESHOOTING

10.1. Проблема 1: Ollama не запускается

Симптомы:

Error: connection refused at <http://localhost:11434>

Решение:

Проверить, запущен ли Ollama:

`tasklist | findstr ollama`

Запустить вручную:

`ollama serve`

Проверить порт:

`netstat -an | findstr 11434`

10.2. Проблема 2: Stable Diffusion — ошибка CUDA

Симптомы:

`RuntimeError: CUDA out of memory`

Решение:

- Уменьшить разрешение: width 1080 → 768, height 1080 → 768
- Уменьшить steps: 30 → 20
- Добавить параметр при запуске: `--medvram`
- Для слабых GPU (4-6GB): `--lowvram`

`.\webui.bat --api --xformers --medvram`

10.3. Проблема 3: n8n не видит localhost-сервисы

Симптомы:

`Error: connect ECONNREFUSED 127.0.0.1:11434`

Решение:

Docker на Windows не видит localhost хоста. Использовать специальный DNS:

- Вместо: <http://localhost:11434>
- Использовать: <http://host.docker.internal:11434>

Обновить все URL в n8n workflow:

- Ollama: <http://host.docker.internal:11434>
- Stable Diffusion: <http://host.docker.internal:7860>
- Flask API: <http://host.docker.internal:5001>

10.4. Проблема 4: DeepSeek генерирует медленно

Симптомы: Генерация текста занимает 5-10 минут

Решение:

1. Проверить использование GPU: `ollama ps` (должно быть GPU)
2. Переустановить драйверы NVIDIA

3. Использовать меньшую модель: ollama pull deepseek-r1:1.5b
4. Для критичных задач использовать YandexGPT (быстрее)

10.5. Проблема 5: Google Sheets API превышает лимиты

Симптомы:

Error 429: Too Many Requests

Решение:

Добавить rate limiting в scripts/data_sync.py:

```
import time

def sync_with_rate_limit():
    for sheet_name, range_name in sheets_config.items():
        print(f'Syncing {sheet_name}...')
        df = get_google_sheet_data(sheet_name, range_name)
        df.to_csv(f'data/{sheet_name}.csv', index=False)
```

```
time.sleep(2) # Пауза 2 секунды
```

Использовать кэширование: синхронизировать только раз в час/день.

11. ЗАКЛЮЧЕНИЕ

11.1. Итоги локализации

Достигнуто:

- Снижение затрат с \$146/мес до \$25/мес (-83%)
- Полный контроль над генерацией контента
- Независимость от зарубежных сервисов
- Приватность данных (локальная обработка)
- Гибкость настройки под специфику проекта

Компромиссы:

- Требуется мощное железо (GPU 12GB+, 16GB RAM)
- Более сложная настройка (Docker, WSL, API-серверы)
- Нет полноценной замены для генерации видео
- Локальные модели могут быть медленнее облачных

11.2. Рекомендуемая конфигурация

Компонент	Решение	Причина
Оркестратор	n8n (Docker)	Бесплатно, полный контроль
LLM (70%)	Ollama DeepSeek	Бесплатно, приватность
LLM (30%)	YandexGPT	Скорость, качество
Изображения	Stable Diffusion	Бесплатно, хорошее качество
Видео	Kling AI (облако)	Нет локальных аналогов
Обработка медиа	FFmpeg + Whisper	Открытые, надежные

Table 9: Финальная архитектура

11.3. Следующие шаги

1. Развернуть локальное окружение по этому гайду
2. Протестировать каждый компонент отдельно
3. Импортировать workflow в n8n
4. Запустить первый цикл генерации контента
5. Оптимизировать промпты и параметры
6. Масштабировать до 100+ публикаций в день

11.4. Полезные ссылки

Документация:

- Ollama: <https://github.com/ollama/ollama>
- n8n: <https://docs.n8n.io/>
- Stable Diffusion WebUI: <https://github.com/AUTOMATIC1111/stable-diffusion-webui>
- YandexGPT: <https://cloud.yandex.ru/docs/yandexgpt/>
- Faster Whisper: <https://github.com/guillaumekln/faster-whisper>

Сообщества:

- n8n Community: <https://community.n8n.io/>
- Stable Diffusion Reddit: <https://www.reddit.com/r/StableDiffusion/>
- Ollama Discord: <https://discord.gg/ollama>

ГАЙД ГОТОВ К ИСПОЛЬЗОВАНИЮ. УСПЕШНОЙ ЛОКАЛИЗАЦИИ КОНТЕНТ-ЗАВОДА!